BAI-2A

COAL LAB 01

K25-0064
SARIM RAZA ANSARI

Part A:

Task 01:

D) in eax 35 will be appear in hexadecimal because 10 + 25 is 35 and it will store in eax.

```
Microsoft Visual Studio Debug Console

  EAX=00000010  EBX=00000025  ECX=009F10FF  EDX=009F10FF
  ESI=009F10FF  EDI=009F10FF  EBP=0133FB10  ESP=0133FB04
  EIP=009F366F  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1


  EAX=00000035  EBX=00000025  ECX=009F10FF  EDX=009F10FF
  ESI=009F10FF  EDI=009F10FF  EBP=0133FB10  ESP=0133FB04
  EIP=009F3676  EFL=00000206  CF=0  SF=0  ZF=0  OF=0  AF=0  PF=1


C:\Users\Administrator\source\repos\COAL - 1\Debug\COAL - 1.exe (process 6120) exited with code 0 (0x0).
Press any key to close this window . . .
```

Task 02:

d) If left on x64

```
A1000    cannot open file : Irvine32.inc
MSB3721 The command "ml64.exe /c /nologo /Zi /Fo"x64\Debug\2.obj" /W3 /errorReport:prompt  /Ta2.asm" exited with code 1.
```

The Irvine32 library is intended for 32-bit x86 MASM programs. If left it will fail to link.

```
Microsoft Visual Studio Debug Console

  EAX=006FF884  EBX=0048D000  ECX=00000025  EDX=00000015
  ESI=009910FF  EDI=009910FF  EBP=006FF838  ESP=006FF82C
  EIP=0099366F  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1


  EAX=006FF884  EBX=0048D000  ECX=00000010  EDX=00000015
  ESI=009910FF  EDI=009910FF  EBP=006FF838  ESP=006FF82C
  EIP=00993676  EFL=00000202  CF=0  SF=0  ZF=0  OF=0  AF=0  PF=0


C:\Users\Administrator\source\repos\COAL - 1\Debug\COAL - 1.exe (process 11832) exited with code 0 (0x0).
Press any key to close this window . . .
```

Task 03:

d) **Step Into (F11):** Executes the current instruction and, when the instruction is a function call, enters that function.

Step Over (F10): Executes the current instruction. If it is a procedure, the debugger executes the procedure without taking you through its internal instructions.

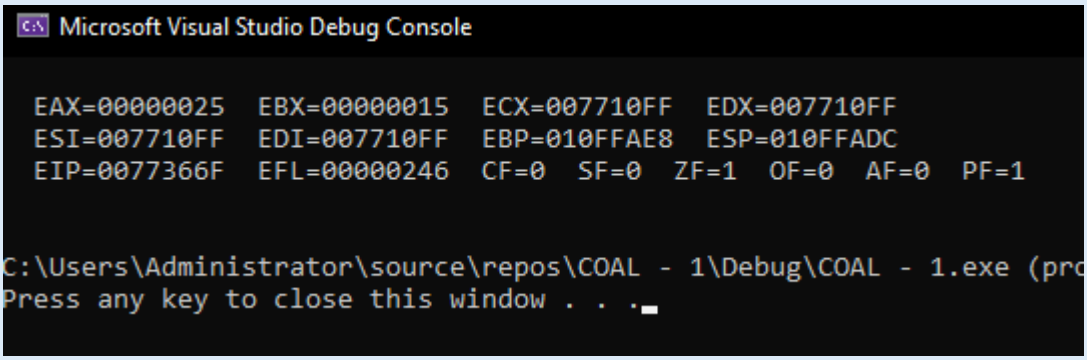
Step Out: Finishes executing the current procedure and returns to the calling code.

```
INCLUDE Irvine32.inc

.code
main PROC
   mov edx, OFFSET msg   ; point to string
   call WriteString      ; print string
   exit
main ENDP

.data
msg BYTE "Hello, World!", 0
END main
```

Task 04:

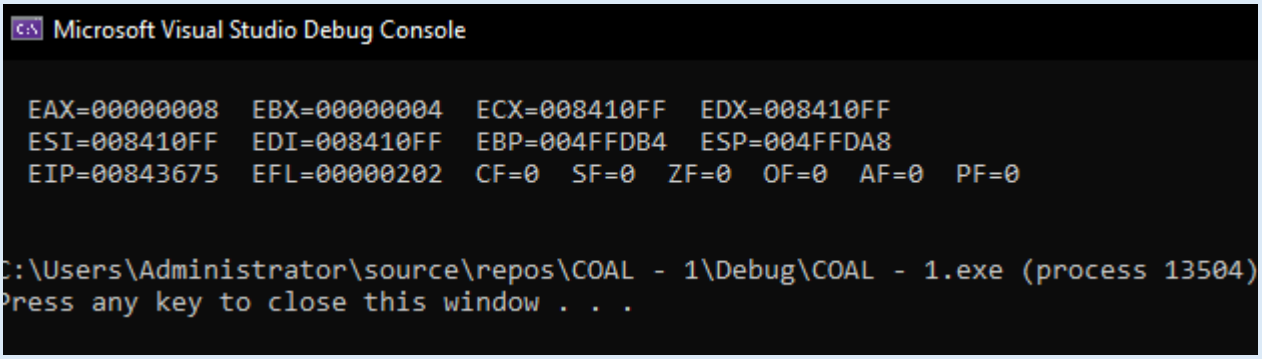


Task 05:

c)

Instruction	Register	before	after
mov eax, 4	EAX	Unknown	4
add eax, 4	EAX	4	8
mov ebx, 9	EBX	Unknown	9
sub ebx, 5	EBX	9	4

d)The initial value of both eax and ebx was unknown then I move 4 and 5 respectively then add and sub 4 and 5 respectively.



Part B:

Task 01:

a) Assembly language is preferred for an embedded system because it provide direct control over computer's hardware and registers. It allow programmers to write highly effective code with minimal space that can run much faster then ordinary program. It also provide precise control over CPU which is useful to respond quickly. Since embedded systems may have limited memory and processing power, avoiding unnecessary overhead can improve performance. Assembly is therefore suitable for critical parts of firmware where efficiency and hardware control are important.

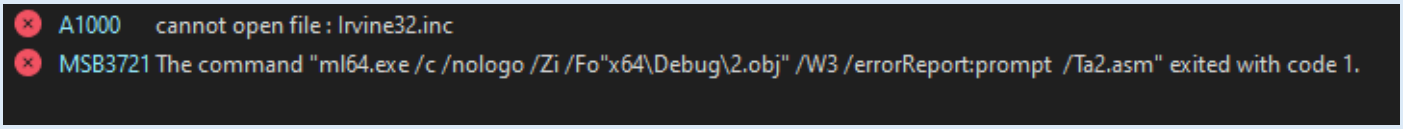
b) Automotive engine control systems. Their embedded system has to respond quickly

Task 02:

a) 1. The Irvine maybe in different directory.
   2. The project maybe configured as x64 instead of x86

b) 1. Right Click file, then properties, then general and at path provide correct path where Irvin is.
   2. Change configuration to x86.

c) If this project were built targeting an x64 architecture, the build would fail or crash because the Irvine32 library is explicitly written for 32-bit protected mode (x86).



Task 03:

a) **1.TITLE:** Gives the Assembly source file a title.
   2. INCLUDE: It will include Irvine32 library

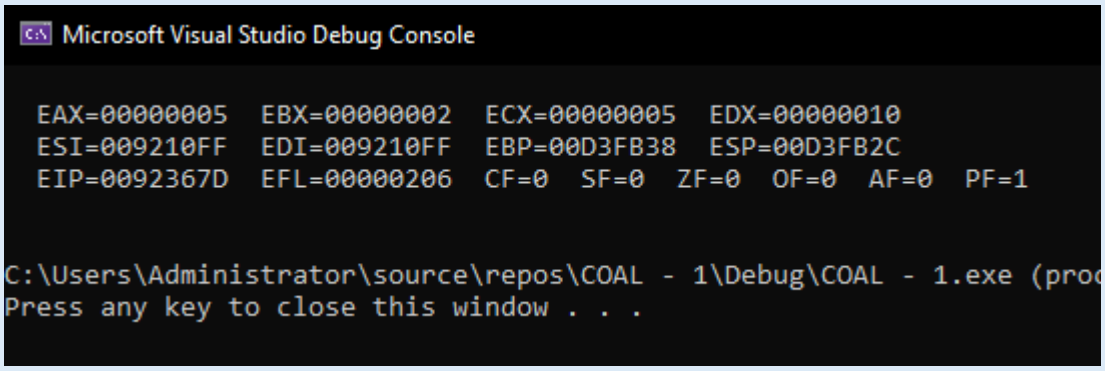
3. .code: Marks the beginning of the code.
4. Main PROC: Marks Beginning of m
5. Mov eax, 10: Copies 10 into register eax.
6. Mov ebx, 25: Copies 25 into register ebx.
6. Call DumpRegs: Displays the register contents.
7. Main ENDP: Marks end of main
8. END main: Tells MASM that main is the program's entry point.

b) Eax = 10 and ebx = 25 and all other register will show garbage values

c) If the project targets x64 instead of x86, the Irvine32 library may not work correctly because the Irvine32 setup is intended for 32-bit x86 programs.

Task 04:

a)



b) First, the programmer saves the Assembly source code as an .asm file. During the assembling stage, MASM translates the Assembly instructions into object code and produces an .obj file. During the linking stage, the linker combines the object file with required libraries such as Irvine32.lib and creates the executable file. During the loading stage, the operating system loads the executable into memory and prepares it for execution. Finally, during execution, the CPU fetches and executes the machine instructions, and the program's output appears on the screen.

Task 05:

a) A breakpoint is a point in a program where the debugger temporarily pauses execution. It is useful because it allows the programmer to inspect the values of registers and other program information at a specific moment. This is also helpful for finding logical errors.

b) F11: The debugger attempts to step into the DumpRegs procedure and allows you to inspect its internal execution.

F10: The debugger executes the entire DumpRegs procedure and stops at the next instruction in your own program.

Step Out: The debugger finishes the current procedure and returns to the instruction after the original call DumpRegs.

c) Observing register values during execution helped me understand how individual Assembly instructions affect the CPU state. For example, after executing mov eax, 10h, I could directly see the value of EAX change to 10h. After executing an add instruction, I could see the new result immediately in the Registers.